

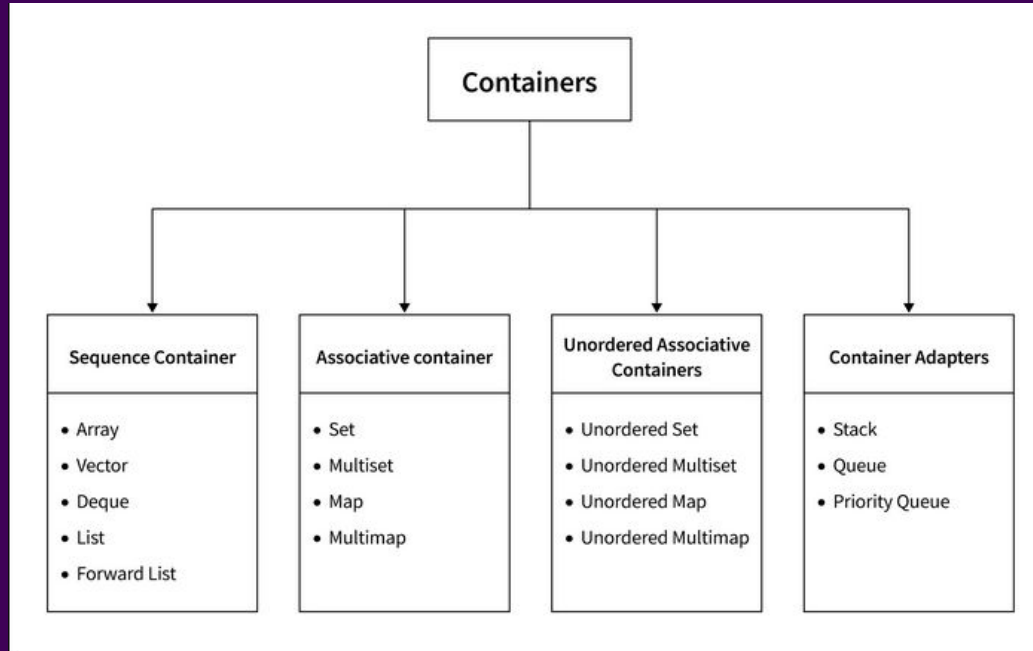
Associative Containers

CSCI 33500: Software Design and Analysis III

Sadab Hafiz
sh3646@hunter.cuny.edu

09

Containers in C++ Standard Template Library (STL)



Note that container adapters are not really a category of containers. Find out more about the STL here:

<https://www.geeksforgeeks.org/cpp/the-c-standard-template-library-stl/>

Container Types

C++ STL Containers

- Sequence Containers
 - ◆ Containers that store individual elements in linear data structures allowing the elements to be accessed sequentially
- Container Adapters
 - ◆ Provides an interface for other containers by adapting the behavior of the existing containers to fit specific use case
 - ◆ For example: stacks using vectors
- Associative Containers
 - ◆ Containers that store keys but also store records associated with those keys
 - ◆ STL provides both ordered and unordered associative containers with different implementations
- Note that C++ STL now has even more containers. Find out [here](#).

Container Types

C++ STL Containers

- Sequence Containers
 - ◆ Containers that store individual elements in linear data structures allowing the elements to be accessed sequentially
- Container Adapters
 - ◆ Provides an interface for other containers by adapting the behavior of the existing containers to fit specific use case
 - ◆ For example: stacks using vectors
- Associative Containers
 - ◆ Containers that store keys but also store records associated with those keys
 - ◆ STL provides both ordered and unordered associative containers with different implementations
- Note that C++ STL now has even more containers. Find out [here](#).

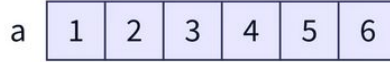
Sequence Containers

Recap

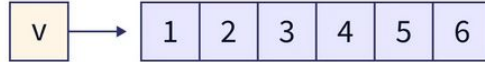
- Linear sequential ordering of data
- Data can be accessed by its position in the sequence (insertion order)
 - ◆ `arr[3]`
 - ◆ `std::next(f_list.begin(), 3);`
- Direct control of inserting and removing an element in sequence
 - ◆ `deq.insert(deq.begin() + 3, 50);`
 - ◆ `vec.erase(vec.begin() + 3);`
- Some allow $O(1)$ random access using contiguous memory:
 - ◆ `std::vector`
 - ◆ `std::array`
 - ◆ `std::deque`

Recap: Sequence Containers

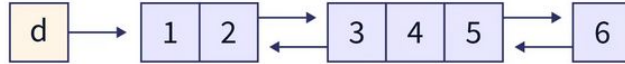
`array<T, n>`



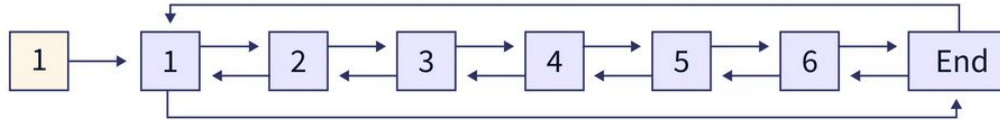
`vector<T>`



`deque<T>`



`list<T>`



`forward_list<T>`



Associative Containers

Introduction

- Non-linear data storage where location of the data is dictated by the data itself (a key)
- Insertion order is not maintained
 - ◆ Sorted Order
 - ◆ Hashing
- The container handles location of data within its own storage
- Doesn't use contiguous memory so no $O(1)$ random access

Associative Containers

Introduction

- Both ordered and unordered variants are provided in C++ STL
- Some store keys only (set):
 - ◆ `std::set`
 - ◆ `std::multiset`
 - ◆ `std::unordered_set`
 - ◆ `std::unordered_multiset`
- Some store key-value pairs (map):
 - ◆ `std::map`
 - ◆ `std::multimap`
 - ◆ `std::unordered_map`
 - ◆ `std::unordered_multimap`
- The “multi” associative containers simply allow duplicate keys

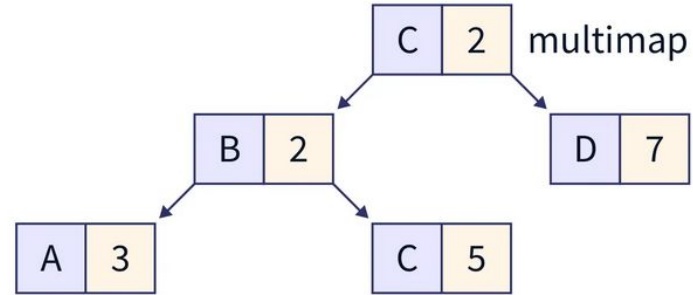
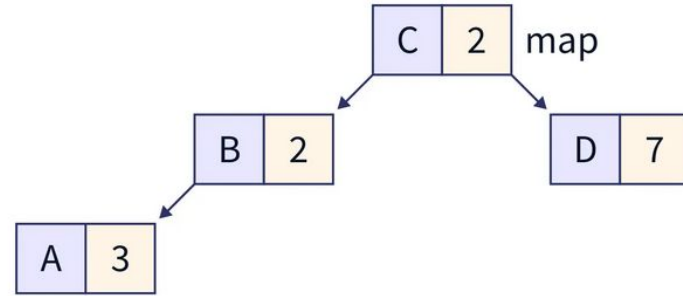
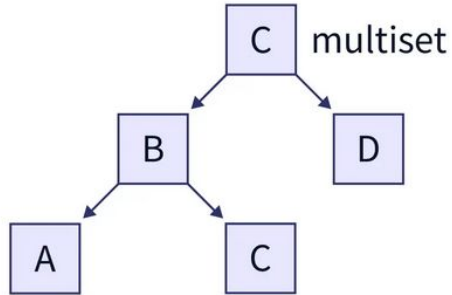
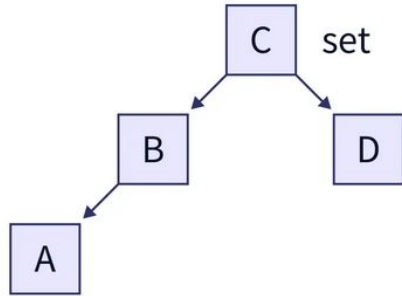
Associative Containers

Ordered Containers

What data structure can we use for the ordered variants?

- Both ordered and unordered variants are provided in C++ STL
- Some store keys only (set):
 - ◆ `std::set`
 - ◆ `std::multiset`
 - ◆ `std::unordered_set`
 - ◆ `std::unordered_multiset`
- Some store key-value pairs (map):
 - ◆ `std::map`
 - ◆ `std::multimap`
 - ◆ `std::unordered_map`
 - ◆ `std::unordered_multimap`
- The “multi” associative containers simply allow duplicate keys

Ordered Associative Containers

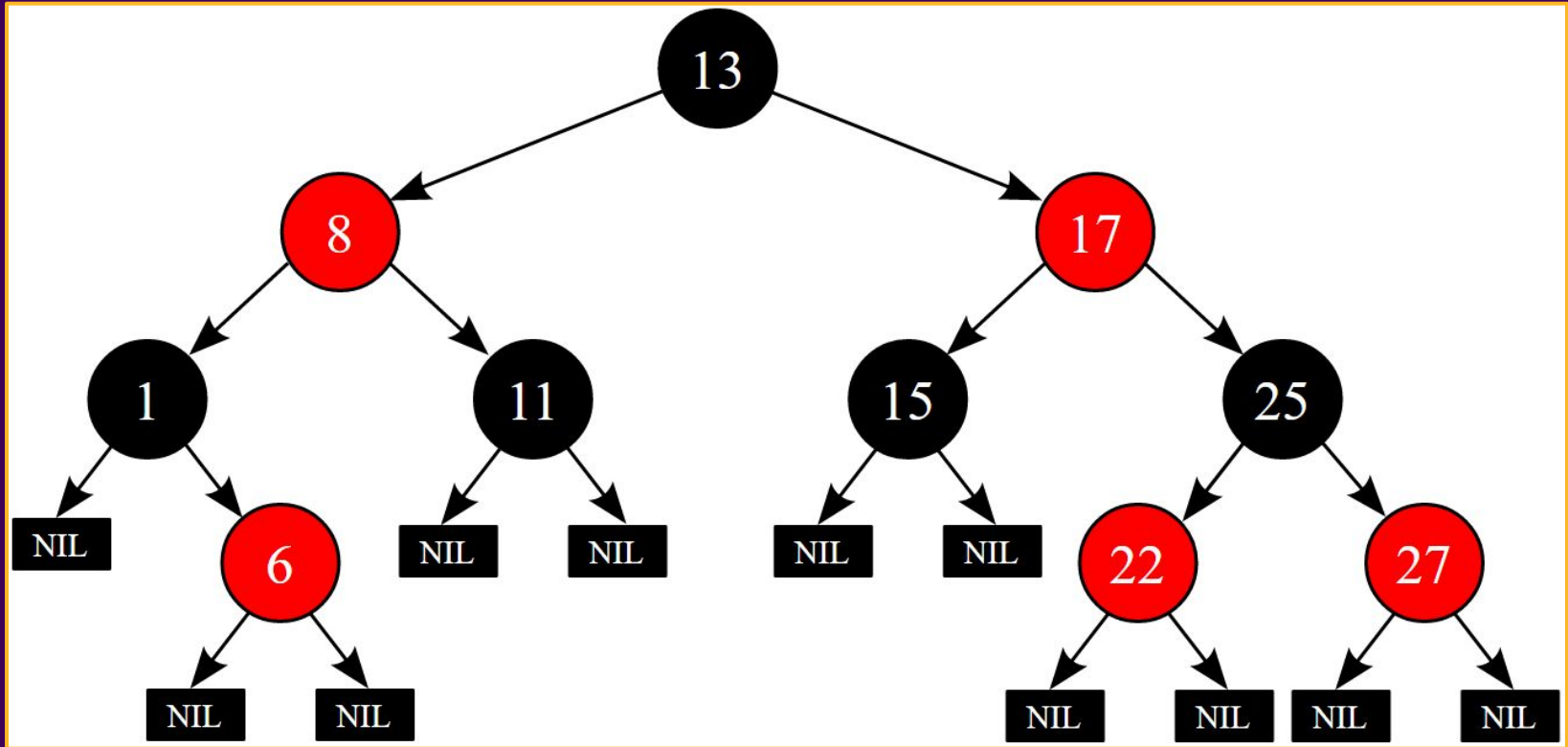


Associative Containers

Ordered Containers

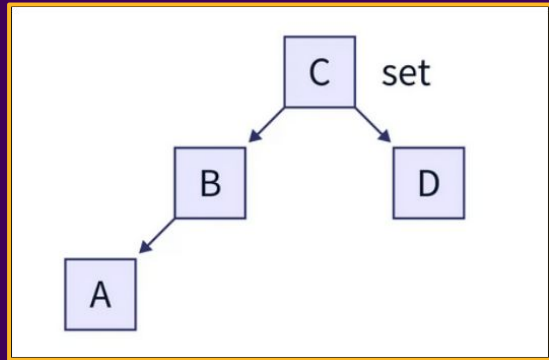
- The standard mandates **$O(\log(n))$** worst-case complexity for:
 - ◆ Lookups
 - ◆ Insertions
 - ◆ Deletions
- All major compilers use Red-Black Trees for implementation
 - ◆ Balanced Binary Search Tree (BST)
 - ◆ Preferred to AVL Trees because it is less expensive to rebalance
 - ◆ More complex structural cases
 - ◆ Color-coded nodes for balance
- All provide Bi-directional Iterator
 - ◆ Traversal is **in-order** (left, root, right)

Red-Black Tree



Set

Associative Container



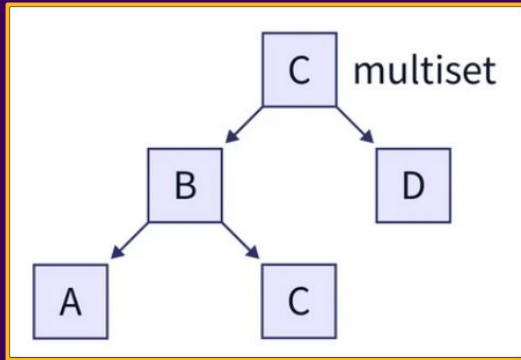
- Container that stores **unique** elements following a specific order
- The data itself is treated as a key
 - ◆ Cannot be modified once in the container
 - Always `const`
 - ◆ Can be inserted and removed
- Properties:
 - ◆ Elements are referenced by the key and not their absolute position in container
 - ◆ Elements follow a strict order at all times
 - ◆ The element's value is treated as the key
 - ◆ No two elements may have same key
 - ◆ Allocates more space as the container grows in size (one element each time)
- [Find out more in the documentation](#)

Usage Example

<https://www.geeksforgeeks.org/cpp/set-in-cpp-stl/>

Multiset

Associative Container



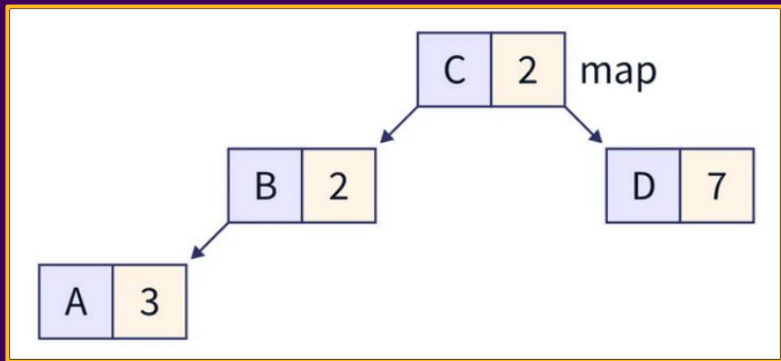
- Container that stores elements following a specific order
- Allows duplicates (equal elements)
- The data itself is treated as a key
 - ◆ Cannot be modified once in the container
 - Always `const`
 - ◆ Can be inserted and removed
- Properties:
 - ◆ Elements are referenced by the key and not their absolute position in container
 - ◆ Elements follow a strict order at all times
 - ◆ The element's value is treated as the key
 - ◆ Multiple elements may have same key
 - ◆ Allocates more space as the container grows in size (one element each time)
- [Find out more in the documentation](#)

Usage Example

<https://www.geeksforgeeks.org/cpp/multiset-in-cpp-stl/>

Map

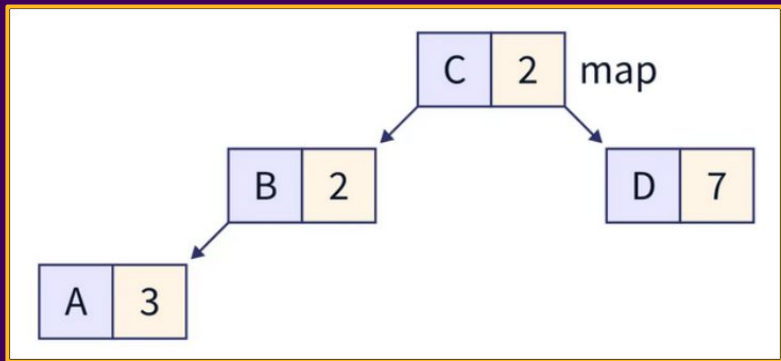
Associative Container



- Container that stores **unique** key-value pairs maintaining a specific order
- How data is structured:
 - ◆ Key: To sort and uniquely identify items
 - Cannot be modified once inserted
 - Always **const**
 - ◆ Value: Data associated with specific key
 - ◆ Key and value may have different types
 - Grouped together using [std::pair](#)
- Basic Operations:
 - ◆ Can be inserted and removed (by key)
 - ◆ Values can be accessed using: **map[key]**
 - Note that using **[]** for accessing a key that doesn't exist leads to insert of key with default value
 - Use **.find()** to verify first

Map

Associative Container



→ Properties:

- ◆ Elements are referenced by the key and not their absolute position in container
- ◆ Elements follow a strict order at all times
- ◆ Each element associates a key (identifier) to a mapped value (main data)
- ◆ No two elements may have same key
- ◆ Allocates more space as the container grows in size (one element each time)

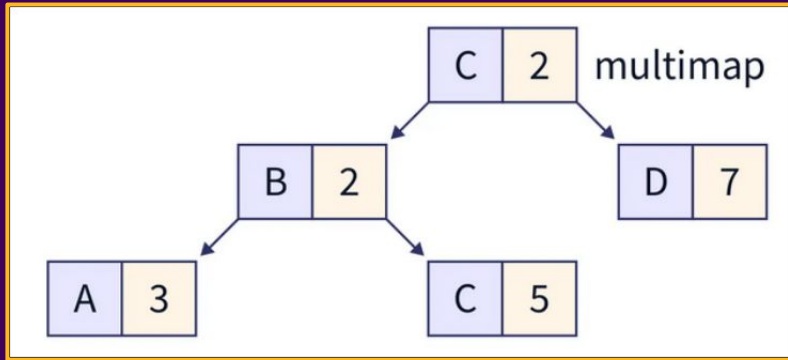
→ [Find out more in the documentation](#)

Usage Example

<https://www.geeksforgeeks.org/cpp/map-associative-containers-the-c-standard-template-library-stl/>

Multimap

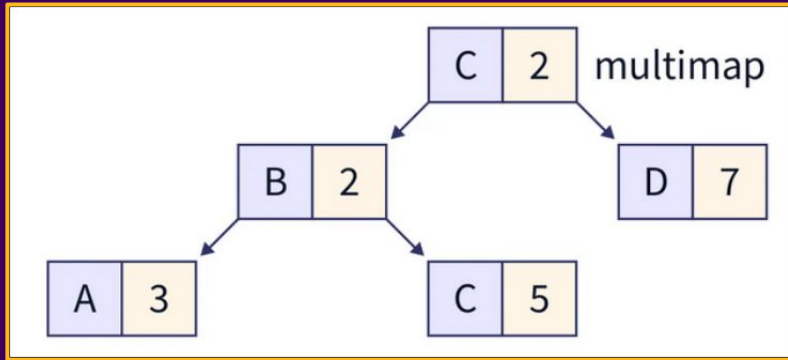
Associative Container



- Container that stores key-value pairs maintaining a specific order
- Allows duplicates (keys)
- How data is structured:
 - ◆ Key: To sort and uniquely identify items
 - Cannot be modified once inserted
 - Always `const`
 - ◆ Value: Data associated with specific key
 - ◆ Key and value may have different types
 - Grouped together using `std::pair`
- Basic Operations:
 - ◆ Can be inserted and removed (by key)
 - ◆ Values can be accessed using iterators:
 - `equal_range(key)`: Returns a pair containing two iterators:
 - `first`: first key match
 - `second`: last key match + 1
 - Can use `.find()` to get an iterator to the first element matching key

Multimap

Associative Container



→ Properties:

- ◆ Elements are referenced by the key and not their absolute position in container
- ◆ Elements follow a strict order at all times
- ◆ Each element associates a key (identifier) to a mapped value (main data)
- ◆ Multiple elements may have same key
- ◆ Allocates more space as the container grows in size (one element each time)

→ [Find out more in the documentation](#)

Usage Example

<https://www.geeksforgeeks.org/cpp/multimap-associative-containers-the-c-standard-template-library-stl/>

Big-O of C++ STL Associative Containers

Operation	Set	Multiset	Map	Multimap
Find				
Insert				
Erase				
Clear				
Copy				
Destructor				
Move				
Iterator (<code>++it</code>)				

→ Time complexity will be the same for all of them

- ◆ Balanced BST

Big-O of C++ STL Associative Containers

Operation	Set	Multiset	Map	Multimap
Find	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Insert	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Erase	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Clear				
Copy				
Destructor				
Move				
Iterator (<code>++it</code>)				

→ Time complexity will be the same for all of them

- ◆ Balanced BST

→ Core operations are $O(\log(n))$

- ◆ Traversing from root to leaf

Big-O of C++ STL Associative Containers

Operation	Set	Multiset	Map	Multimap
Find	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Insert	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Erase	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Clear	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Copy	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Destructor	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Move				
Iterator (++it)				

- Time complexity will be the same for all of them
 - ◆ Balanced BST
- Core operations: $O(\log(n))$
 - ◆ Traversing from root to leaf
- Memory management: $O(n)$
 - ◆ Handling every node

Big-O of C++ STL Associative Containers

Operation	Set	Multiset	Map	Multimap
Find	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Insert	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Erase	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Clear	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Copy	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Destructor	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Move	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Iterator (<code>++it</code>)				

- Time complexity will be the same for all of them
 - ◆ Balanced BST
- Core operations: $O(\log(n))$
 - ◆ Traversing from root to leaf
- Memory management: $O(n)$
 - ◆ Handling every node
- Move operations: $O(1)$
 - ◆ Transferring ownership

Big-O of C++ STL Associative Containers

Operation	Set	Multiset	Map	Multimap
Find	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Insert	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Erase	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Clear	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Copy	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Destructor	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Move	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Iterator ($++it$)	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(1)^*$

- Time complexity will be the same for all of them
 - ◆ Balanced BST
- Core operations: $O(\log(n))$
 - ◆ Traversing from root to leaf
- Memory management: $O(n)$
 - ◆ Handling every node
- Move operations: $O(1)$
 - ◆ Transferring ownership
- Iterator($++it$): $O(1)^*$
 - ◆ In-order traversal
 - Individual steps vary
 - Full traversal is $O(n)$
 - ◆ Ascending order data access

After Hash Tables

Unordered Containers

- Both ordered and unordered variants are provided in C++ STL
- Some store keys only (set):
 - ◆ `std::set`
 - ◆ `std::multiset`
 - ◆ `std::unordered_set`
 - ◆ `std::unordered_multiset`
- Some store key-value pairs (map):
 - ◆ `std::map`
 - ◆ `std::multimap`
 - ◆ `std::unordered_map`
 - ◆ `std::unordered_multimap`
- The “multi” associative containers simply allow duplicate keys